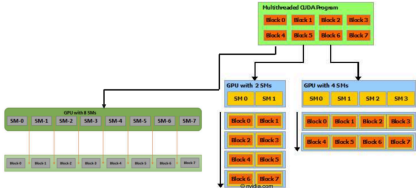


GPU (GRAPHICS PROCESSING UNIT)

End of Moores Law: We can no longer gain performance by "growing" sequential processors. Instead, we improve performance by running code in parallel on multiple **CPUs** (*low latency*) and many-core or **massively parallel co-processors (GPUs)** (*high throughput*). GPUs are specialized electronic circuits designed to accelerate the computation of computer graphics. They are faster than CPUs for suitable algorithms on large datasets. Useful for calculations which consist of multiple **independent sub-calculations**, not very useful for calculations where the results rely on the previous results (*line fibonaccis*). **High Parallelization:** A CPU offers few cores (4, 8, 16, 64) and is very fast. Programming is easier. A GPU offers a very large number of cores (512, 1024, 3824, 5760) and has very specific slower processors. It is optimized for throughput. Programming is more difficult. **GPU Structure:** A GPU consists of multiple **Streaming Multiprocessors (SM)** which in turn consist of multiple **Streaming Processors (SP)** (e.g. 1-30.2M, 6-152 SPs per SM). **SIMD:** Single Instruction Multiple Data. The same instruction is executed simultaneously on multiple cores working on different data elements (*vector parallelism*). Saves fetch & decode instructions. **SUD:** Single Instruction Single Data. Purely sequential calculations. **SIMT:** Single Instruction Multiple Threads. The same instruction is executed in different threads over different data.



LATENCY VS. THROUGHPUT

Latency: Elapsed time of an event (*Walking from point A to B takes one minute, the time is minute*). **Throughput:** The number of events that can be executed per unit of time (*bandwidth*). There is a **tradeoff** between latency and throughput. Increasing throughput by pipelined processing, latency must often also increase. All pipeline stages must operate in lockstep. The rate of processing is determined by the slowest step. **Pipelining:** Run processes in an overlapping manner. **Example:** A program consists of two operations: Transfer data from CPU memory to GPU memory (T_1 units = 20ms). Execute computation on the device (T_2 units = 60ms). What is the **latency** (non-pipelined)? $20 + 60 = 80ms$. What is the **throughput** (pipelined)? Every 60ms an operation is finished. **Throughput** = $1/60$ operations/ms.

CPU VS GPU

CPUs	GPUs
- Low latency - Fast at lot optimized cores - General purpose - Architecture and Compiler help - to run any code fast	- Can execute highly parallel data operations - Simple but a lot of cores with cache per core - very useful for problems which consist of a lot of independent data elements - Efficiency must be achieved by optimizing the program
Aim: low latency per thread	Aim: high throughput

NUMA MODEL

NUMA stands for Non-Uniform Memory Access. CPUs on host and GPU devices each have local memories. There is no common main memory between the two, so explicit transfer between CPU and GPU is needed. There is also no garbage collector on the GPU.

CUDA

Computer Unified Device Architecture. Is a parallel computing platform and an API for Nvidia GPU that allows the host program to use GPUs for general purpose processing.

CUDA Execution steps

- cudaLaunch: GPU memory allocate
- cudaMemcpy: Data transfer to GPU (hostToDevice)
- Kernel<=0, N>: kernel execution
- cudaFree: Deallocate GPU memory

Example: Array addition

```
int i = 0; i < N; i++) { c[i] = A[i] + B[i]; } // sequential
int i = 0; i < N; i++) { c[i] = A[i] + B[i]; } // parallel using n threads
```

CUDA Kernel

A kernel is a function that is executed n times in parallel by m different CUDA threads.

```
// kernel definition on GPU
__global__
void VectorAddKernel(float* A, float* B, float* C) {
    int i = threadIdx.x; C[i] = A[i] + B[i];
}
```

// kernel invocation on CPU

```
VectorAddKernel(<N>, <N>, A, B, C); // N is amount of threads
Only the GPU knows when the task is finished.
```

Boilerplate Orchestration Code

```
void cudaVectorAdd(float* h_A, float* h_B, float* h_C, int N) {
    size_t size = N * sizeof(float);
    float* d_A, *d_B, *d_C; // data on GPU
    cudaMalloc((void**)&d_A, size); cudaMalloc((void**)&d_B, size);
    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);
    VectorAddKernel<<N>>>(d_A, d_B, d_C, N);
    cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);
    cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
}
```

PERFORMANCE METRICS

The performance is either limited by **memory bandwidth** or **computation**. **Compute Bound:** Throughput is limited by calculation (*Cores are at the limit, but memory bandwidth can transfer more data*). This is better and reached if AI kernel = AI GPU. **Memory Bound:** Throughput is limited by data transfer (*Memory bound because it is limited by how fast data can be moved from host to device*). **Arithmetic intensity:** Defined as FLOPS (Floating Point Operations per Second) per Byte. The higher, the better. $\frac{\text{Number of operations}}{\text{Number of transferred bytes}} = \text{FLOPS}$

Example: For $(i=0; i < N; i++) \{ x[i] = x[i] + y[i] + x[i]; \}$ Read x and y from memory, write z to memory. That's 2 reads and 1 write (a is used twice but read only once). In case x and z are ints, we have 12 (4 + 4) bytes transferred and 2 arithmetic ops (+, +). The arithmetic intensity is therefore $\frac{2}{12} = \frac{1}{6}$.

Compute Bound

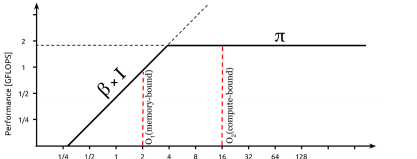
$$\begin{aligned} t_{\text{compute}} &> t_{\text{memory}} \\ \frac{\text{ops}}{BW_{\text{compute}}} &> \frac{\text{bytes}}{BW_{\text{memory}}} \\ \frac{\text{ops}}{\text{bytes}} &> \frac{BW_{\text{compute}}}{BW_{\text{memory}}} \end{aligned}$$

Memory Bound

$$\begin{aligned} t_{\text{compute}} &< t_{\text{memory}} \\ \frac{\text{ops}}{BW_{\text{compute}}} &< \frac{\text{bytes}}{BW_{\text{memory}}} \\ \frac{\text{ops}}{\text{bytes}} &< \frac{BW_{\text{compute}}}{BW_{\text{memory}}} \end{aligned}$$

Roofline model

Provides performance estimates of a kernel running on differently sized architectures. Has three parameters: Peak performance, peak bandwidth vs. arithmetic intensity. **Peak performance** is derived from benchmarking FLOPS or GLOPS (*ops/FLOPS*, 10^9 FLOPS). The peak bandwidth from manuals of the memory subsystem. The ridge point where the horizontal and diagonal lines meet = minimum AI required to achieve the peak performance.



GPU ARCHITECTURE

Because there are so many cores on GPUs, it is possible to run many threads in parallel without context switches. This allows better parallelism without a performance penalty.

COMPIATION

Just-in-time Compilation: The NVCC compiler compiles the non-CUDA code with the host C compiler and translates code written in CUDA into PTX instructions (assembly language represented as code). The graphics driver compiles the PTX into executable binary code. The assembly of PTX code is postponed until application runtime, at which time the target GPU is known. The disadvantage of this is the increased application startup delay. However, thanks to cache this only happens once (*warmup*).

Programming Interface: Runtime (The cudart library provides functions that execute on the host-to-do-jalloc device memory, transfer data etc.) **or Driver API** (The CUDA driver API is implemented in the cuda.dll or cuda.so which is copied on the system during installation of the driver. This provides an additional level of control by exposing low-level concepts such as CUDA context, CUDA kernel).

Asynchronous Execution: The command pipeline in CUDA works asynchronous, commands and data can be transferred from/to the GPU at the same time.

CUDA SIMT EXECUTION MODEL

Single instruction, multiple Threads. The kernel is executed N times in parallel by N different CUDA threads.

Blocks: Threads are grouped in blocks. The block can define how many threads there are (up to 1024). Threads in one block can interact with each other but not with threads in other blocks.

Execution Model: One thread runs on one virtual scalar processor (one GPU core). One block runs on one virtual multi-processor (one GPU Streaming Multiprocessor). Blocks must be independent.

Thread Pool Abstraction: The compiled CUDA program has a larger GPU blocks. The runtime can choose how to allocate these blocks to multiprocessors. For a larger GPU with 8 SMs, each SM gets one CUDA block. This enables performance scalability without code changes.

Guarantees: CUDA guarantees that all threads in a block run on the same SM at the same time and that the blocks in a kernel finish before any block from a new, dependent kernel is started.

Mapping: One SM can run several concurrent blocks depending on the resources needed. A kernel is executed on one device. CUDA supports running multiple kernels on the device at one time.

CUDA KERNEL SPECIFICATION

Specifying Kernel: VectorAddKernel<<GRID_dimension, BLOCK_dimension>>>(A,B,C) Dimensions can be 1D, 2D or 3D and specified via dim3 which is a structure designed for storing block and grid dimensions: struct dim3 { x, y, z; };

Coalesced Access: VectorAddKernel<<1,1,1>>> creates 1 thread per block. VectorAddKernel<<dim3(16,16,16)>>> creates 1 thread per block. VectorAddKernel<<dim3(16,16,16)>>> creates 1 thread per block.

Number of blocks in a grid: dim3(x, y, z); dim3(x, y, z); dim3(x, y, z);

Number of threads in a block: dim3Block(x, y, z); dim3Block(x, y, z);

10 Grid: We can simply use integers. VectorAddKernel<<1,1,1>>> creates 1 block with N threads. 10 Grid: dim3 grid(3,3,3); dim3 block(3,3,3); VectorAddKernel<<grid, block>>>

3D Grid: dim3 grid(3,3,3); dim3 block(3,3,3); VectorAddKernel<<grid, block>>>

Device Limits: Max threads per block: 1024; Max thread dimensions per block: (1024, 1024, 64) Max grid size: (2^14, 2^14, 2^14); Max threads per block: 1024, 1024, 64

Calculation Examples: VectorAddKernel<<dim3(4,4,2), dim3(16,16)>>>(d_A, d_B, d_C);

Amount of Blocks: 4 - 2 = 64

Amount of threads per block: 16 - 16 = 256

Threads in total: 64 - 256 = 16384

If we have 1024 threads in the block, how many blocks are needed to launch N threads?

$\text{blocksPerGrid} = \frac{N + \text{threadsPerBlock} - 1}{\text{threadsPerBlock}}$

Devices limits: Max threads per block: 1024; Max thread dimensions per block: 1024, 1024, 64

Index based on (blockID, threadIdx): $\text{blockID} \times \text{blockSize} + \text{threadIdx}$

Index based on (blockID, threadIdx): $\text{blockID} \times \text{blockSize} + \text{threadIdx}$

Index based on (blockID, threadIdx): $\text{blockID} \times \text{blockSize} + \text{threadIdx}$

Index based on (blockID, threadIdx): $\text{blockID} \times \text{blockSize} + \text{threadIdx}$

Index based on (blockID, threadIdx): $\text{blockID} \times \text{blockSize} + \text{threadIdx}$

Index based on (blockID, threadIdx): $\text{blockID} \times \text{blockSize} + \text{threadIdx}$

Index based on (blockID, threadIdx): $\text{blockID} \times \text{blockSize} + \text{threadIdx}$

Index based on (blockID, threadIdx): $\text{blockID} \times \text{blockSize} + \text{threadIdx}$

Index based on (blockID, threadIdx): $\text{blockID} \times \text{blockSize} + \text{threadIdx}$

Index based on (blockID, threadIdx): $\text{blockID} \times \text{blockSize} + \text{threadIdx}$

Index based on (blockID, threadIdx): $\text{blockID} \times \text{blockSize} + \text{threadIdx}$

Index based on (blockID, threadIdx): $\text{blockID} \times \text{blockSize} + \text{threadIdx}$

Index based on (blockID, threadIdx): $\text{blockID} \times \text{blockSize} + \text{threadIdx}$

GPU PERFORMANCE OPTIMIZATIONS

Hardware: A scalable array of multithreaded Streaming Multiprocessors (SMs), the threads of a thread block execute concurrently on one multiprocessor, multiple thread blocks can execute concurrently on one multiprocessor. When thread blocks terminate, new blocks are launched on the free multiprocessors.

MATRIX ADDITION

```
__global__
void MatrixAddKernel(float* A, float* B, float* C) {
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    if (row < A_ROWS && col < A_COLS) { // boundary checking
        C[row * A_COLS + col] = A[row * A_COLS + col] + B[row * A_COLS + col];
    }
}

const int A_ROWS, B_ROWS, C_ROWS = 6;
const int A_COLS, B_COLS, C_COLS = 6;
dim3 block(2,2); dim3 grid(3,3); // (3,2) = (3,2)
MatrixAddKernel<<grid,block>>>(A,B,C);
```

MATRIX MULTIPLICATION

Parallelization: Every thread computes one element of the result matrix C . Can be parallelized because results do not depend on each other.

```
__global__
void MatrixMultiply(float* A, float* B, float* C) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;
    if ((i < N) && (j < N)) { // boundary checking
        float sum = 0;
        for (int k = 0; k < K; k++) {
            sum += A[i * K + k] * B[k * N + j];
        }
        C[i * N + j] = sum;
    }
}
```

MAPPING THREADS / BLOCKS TO GPU WARPS

Warps: Blocks are split into groups (1 Warp = 32 threads) and all threads within execute the same code. If there aren't enough threads to fill a warp, "empty" threads are launched. A number of warps constitutes a thread block. A number of thread blocks are assigned to a Streaming Multiprocessor.

The whole GPU consists of several SMs. Thread blocks are scheduled in parallel or sequentially. Once a thread block is launched on a SM, all of its threads are resident until their execution finishes. Therefore, a new block on a SM is not launched until there is a sufficient number of free registers and shared memory for all warps of the new block.

Warp Execution: All threads in a warp execute the same instruction (SIMD). ASM can accommodate all warps of a block, but only a subset is running in parallel at any time (i to 24).

Divergence: Not all threads of a warp may branch the same way. The branches do not run simultaneously, so threads need to wait until one branch is finished. So branches within one warp should be avoided because of performance problems (branches are born at 1/16 switch...).

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

Example: $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$ $\text{if } (i \< N) \{ \dots \}$

MPMD: Also a "high level" programming model. Multiple Program (Tasks may execute different programs simultaneously), Multiple Data (All tasks may use different data).

Hybrid Memory Model: All processors in a machine can share the memory. They also can request data from other computers. (non-uniform memory access: not all accesses take the same time)

Message Passing Interface (MPI): Distributed programming model. Is a common choice for Parallelization on a cluster, Industry-Standard libraries for multiple programming languages.

MPI Model: Notion of processes (Process is the running program plus its data), parallelism is achieved by running multiple processes, co-operating on the same task. Each process has direct access only to its own data (variables are private). Inter-Process-Communication by sending and receiving messages.

SPMD in MPI: All processes run their own local copy of the program & data. Each process has a unique identifier. Processes can take different paths through the program depending on their IDs. Usually, one process per core is used to maximize the benefit of parallelism.

Formalizing Message: A message transfers a number of data items from the memory of one process to the memory of another process (Typically contains ID of sender and receiver, data type to be sent, number of data items, data itself, message type identifier).

Communication modes: Point to Point (very simple, one sender and one receiver. Relies on matching send and receive calls) and Collective communications (between groups of processes. Broadcast: one to all; Scatter: Split data and send each chunk to different node. Gather: Collect the chunks back at the originating node).

MPI BOILERPLATE CODE

```
int main(int argc, char* argv[]) {
    MPI_Init(&argc, &argv); // MPI Initialization
    int rank; int len;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank); // Process Identification
    char name[MPI_MAX_PROCESSOR_NAME];
    MPI_Get_processor_name(name, &len);
    printf("MPI process %i on %s\n", rank, name);
    MPI_Finalize(); // MPI Finalization
    return 0;
}
```

<